

ОПИСАНИЕ ИНДИВИДУАЛЬНОГО ПРОЕКТА

Дисциплина: «Распределённые и облачные базы данных»

Поле	Значение
ФИО	Ермаков Лаврентий
Группа	М09-КИИ26
Дата	17.09.2026

1. Тема проекта

«Горизонтально фрагментированная СУБД контура исполнения отгрузок на распределённых складских площадках».

Простыми словами: нужно сделать учебный стенд, где склады в разных регионах ведут остатки и заявки на отгрузку на своих узлах PostgreSQL. Это не интернет-магазин и не витрина для покупателя — в центре внимание оператора площадки: что есть на складе, что зарезервировано, что собрали и отправили.

2. Зачем такой проект (идея)

В реальной логистике оперативные данные «привязаны» к площадке: склад WEST не должен писать в остатки EAST. Распределение здесь нужно не «для балансировки ради скорости», а чтобы соблюсти периметр региона, дать автономность при обрыве канала и разделить роли узлов (запись / чтение) внутри региона.

3. Что будет в базе (контур данных)

В учебном стенде два региона — WEST и EAST (контейнеры Docker). Ниже — сущности предметной области и их английские имена в схеме БД.

Таблица (EN)	По-русски	Что хранит
regions	регионы	Периметр площадки: код WEST/EAST, название
warehouses	склады	Склад внутри региона (code, name)
products	товары / номенклатура	Общий справочник: sku, title
stock_balances	остатки	Сколько доступно (qty_available) и в резерве (qty_reserved)
shipment_orders	заявки на отгрузку	Наряд: склад, region_code, status, дата
shipment_items	позиции заявки	Какие товары и в каком количестве (qty)
shipment_events	журнал событий	Смена статуса: from→to, время, кто

3.1. Словарь ключевых полей

Поле (EN)	Перевод / смысл
region_code	код региона — ключ горизонтальной фрагментации
sku	артикул товара (Stock Keeping Unit)
qty_available / qty_reserved	доступно к продаже / уже зарезервировано
status	статус жизненного цикла заявки
from_status / to_status	предыдущий и новый статус в журнале
primary / replica	узел записи / узел чтения (реплика)
async streaming	асинхронная потоковая репликация WAL

4. Почему база именно распределённая

- Периметр площадки. Остатки и заявки WEST не лежат и не меняются вместе с EAST.
- Автономность. При потере канала до другого региона площадка продолжает локальные операции на своём primary.
- Роли узлов. Primary пишет, replica читает. Второй регион — другой фрагмент данных, а не копия «для балансировки».

Критерий распределения: по контуру / площадке (+ режим доступа: оперативка не покидает периметр), ключ размещения — `region_code`.

5. Как данные режутся между узлами

Строка оперативных таблиц с `region_code = WEST` существует только на узлах WEST; то же для EAST. Позиции и события — производная фрагментация: следуют за заявкой своего региона. Справочник `products` копируется на оба региона (реплика справочника).

Параметр	Значение
Критерий	по контуру / площадке + режим доступа (<code>region_code</code>)
Фрагментация	горизонтальная; <code>items/events</code> — производные от заявки
Репликация	async streaming primary → replica внутри региона
Узлы стенда	≥ 4 PostgreSQL (2 региона × primary + replica)
Полнота / непересечение	WEST+EAST = весь контур; одна заявка — один регион

5.1. Если узел упал

- упал primary региона → запись в этот регион нельзя, чтение — с replica;
- упал replica → запись на primary продолжается, чтение можно с primary;
- запрос «сменить статус заявки чужого региона» → отказ, даже если другой узел жив.

6. Операции сверх обычного CRUD

CRUD по складам/товарам/остаткам/заявкам будет. Главное — предметный сценарий отгрузки с резервом остатка и журналом контроля:



Рисунок — жизненный цикл `shipment_orders.status`

- RESERVED — резерв под заявку (нельзя зарезервировать больше `qty_available`);
- PICKING → SHIPPED → DELIVERED — сборка, отправка, подтверждение;
- CANCELLED — отмена из CREATED/RESERVED с откатом резерва;
- каждая смена статуса → запись в `shipment_events`;
- попытка трогать чужой регион → отказ с понятной причиной;
- в логе/UI видно: регион операции и узел (primary / replica).

7. UML / ER предметной области

Модель в ЗНФ: связи через FK, без дублирования названия региона в каждой строке остатка. На диаграмме английские имена таблиц подписаны по-русски.

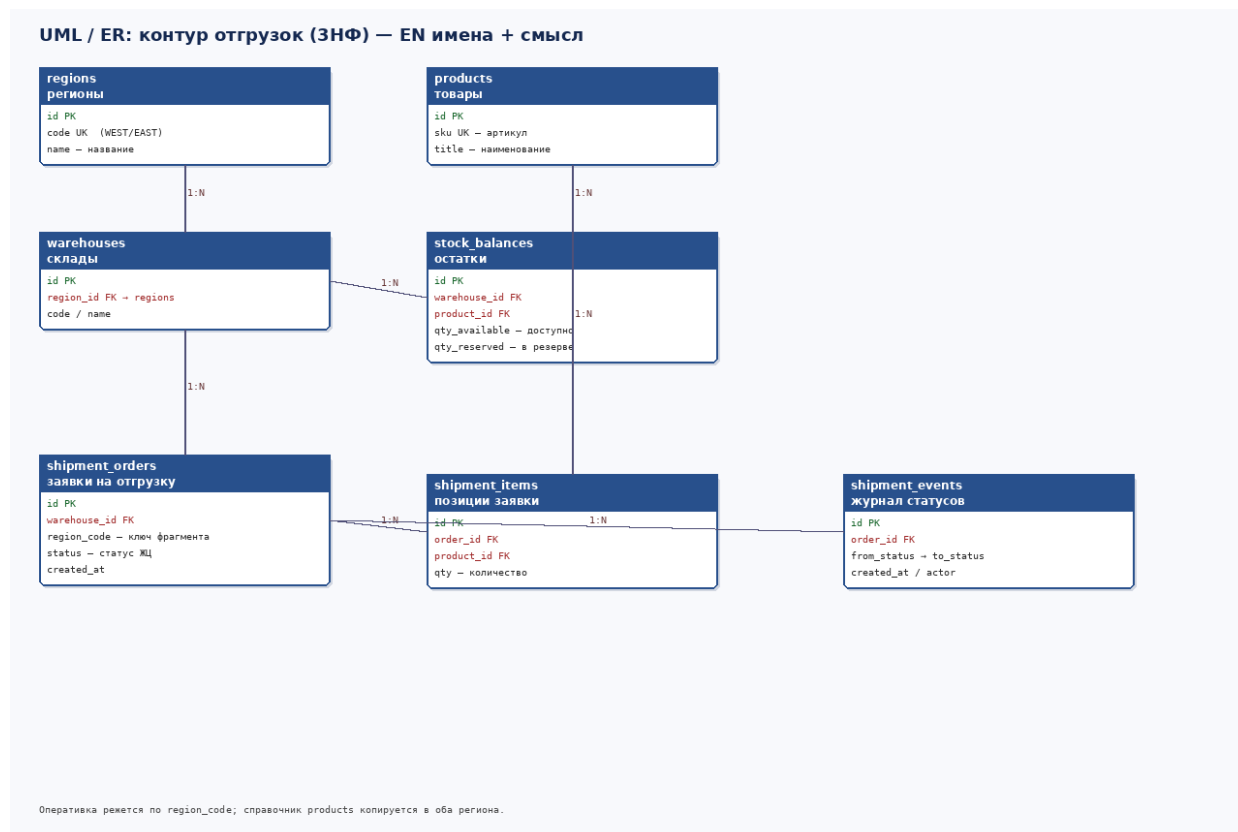


Рисунок 1 — ER-диаграмма (EN + перевод)

```

Регион 1—<N Склад 1—<N Остаток >N—1 Товар
      |
      |—<N Заявка 1—<N Позиция >N—1 Товар
      |
      |—<N Событие (смена статуса)

```

8. UML распределения по узлам

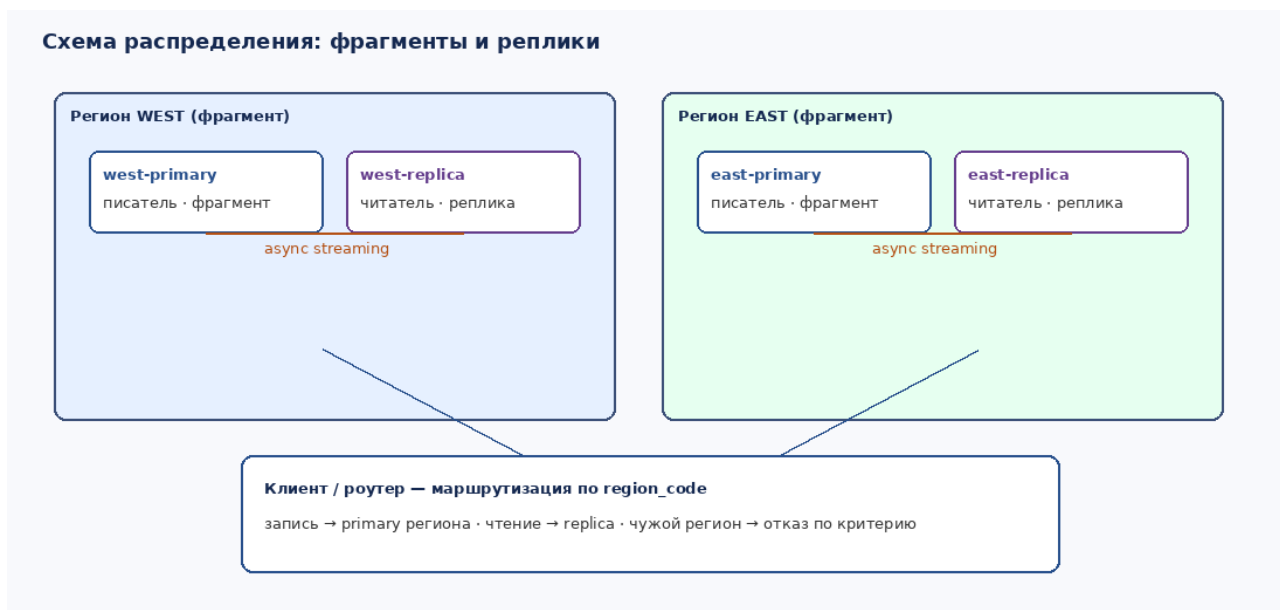


Рисунок 2 — фрагменты регионов и реплики

Узел	Роль	Что хранит	Кто пишет
west-primary	писатель	оперативка WEST	клиент

west-replica	читатель	реплика WEST	никто
east-primary	писатель	оперативка EAST	клиент
east-replica	читатель	реплика EAST	никто

9. Стенд и результат работы

Docker Compose: 4 × PostgreSQL + клиент FastAPI. Запись → primary региона; чтение → replica. Артефакты: docker-compose, SQL/seed, клиент, зависимости, скрипты демо и реальные скриншоты Swagger/API. Каталог реализации: Индивидуальный_проект/.

- схема БД (ЗНФ + FK) и тестовые данные;
- стенд из четырёх узлов (2 региона × primary + replica);
- клиент: полный CRUD warehouses/products/stock/orders + цепочка статусов + отказ узла / отказ по периметру;
- отчёт/описание: UML, схема распределения, демо, скриншоты.

10. Демонстрация стенда (реальные скриншоты)

Ниже — снимки живого стенда (Swagger UI и JSON-ответы клиента), снятые Playwright после прогона `scripts/demo.sh`.

10.1. Стенд: `docker compose ps` — 4 узла PostgreSQL

docker compose ps — 4 узла PostgreSQL						
NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
rdb-east-primary	postgres:16	"docker-entrypoint.s..."	east-primary	25 hours ago	Up 11 minutes (healthy)	0.0.0.0:15434->5432/tcp, [::]:15434->5432/tcp
rdb-east-replica	postgres:16	"/bin/bash /replica-..."	east-replica	25 hours ago	Up 11 minutes (healthy)	0.0.0.0:15435->5432/tcp, [::]:15435->5432/tcp
rdb-west-primary	postgres:16	"docker-entrypoint.s..."	west-primary	25 hours ago	Up 8 minutes (healthy)	0.0.0.0:15432->5432/tcp, [::]:15432->5432/tcp
rdb-west-replica	postgres:16	"/bin/bash /replica-..."	west-replica	25 hours ago	Up 11 minutes (healthy)	0.0.0.0:15433->5432/tcp, [::]:15433->5432/tcp

Рисунок 10.1 — 00_docker_compose_ps.png

10.2. Swagger UI: полный список API (включая `warehouses/products`)

RDB Warehouse Client GET POST	
CRUD + lifecycle операции с реляционными ресурсами	
default	
GET	/health Health
GET	/api/(region)/warehouses List Warehouses
POST	/api/(region)/warehouses Create Warehouse
PUT	/api/(region)/warehouses/{warehouse_id} Update Warehouse
DELETE	/api/(region)/warehouses/{warehouse_id} Delete Warehouse
GET	/api/(region)/products List Products
POST	/api/(region)/products Create Product
PUT	/api/(region)/products/{product_id} Update Product
DELETE	/api/(region)/products/{product_id} Delete Product
GET	/api/(region)/stock List Stock
POST	/api/(region)/stock Create Stock
PUT	/api/(region)/stock/{stock_id} Update Stock
DELETE	/api/(region)/stock/{stock_id} Delete Stock
GET	/api/(region)/orders List Orders
POST	/api/(region)/orders Create Order
POST	/api/(region)/orders/{order_id}/status Change Status
GET	/api/(region)/orders/{order_id}/events Order Events
POST	/api/cross-region-denied Cross Region Denied
Schemas	
HTTPValidationError > Expand all request	
OrderCreate > Expand all request	
ProductCreate > Expand all request	
ProductUpdate > Expand all request	
StatusChange > Expand all request	
StockCreate > Expand all request	
StockUpdate > Expand all request	
ValidationError > Expand all request	
WarehouseCreate > Expand all request	
WarehouseUpdate > Expand all request	

Рисунок 10.2 — 01_swagger_home.png

10.3. CRUD складов: POST /warehouses → primary WEST

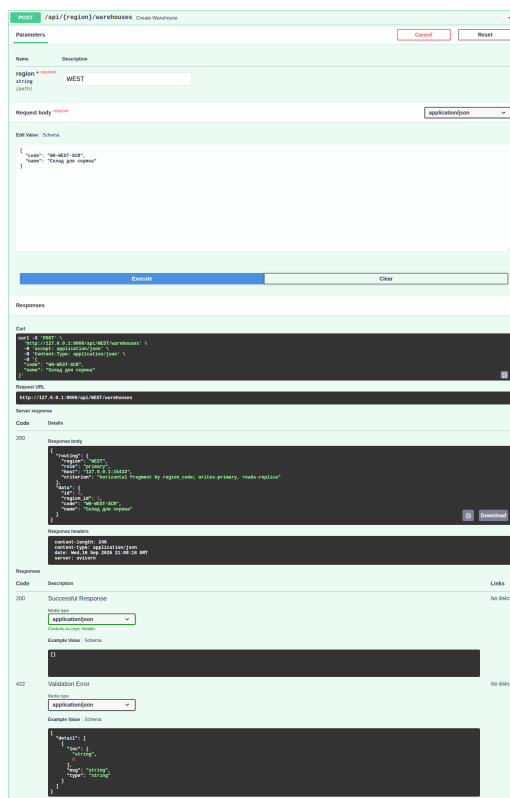


Рисунок 10.3 — 05_swagger_warehouse_post.png

10.4. CRUD товаров: POST /products → primary WEST

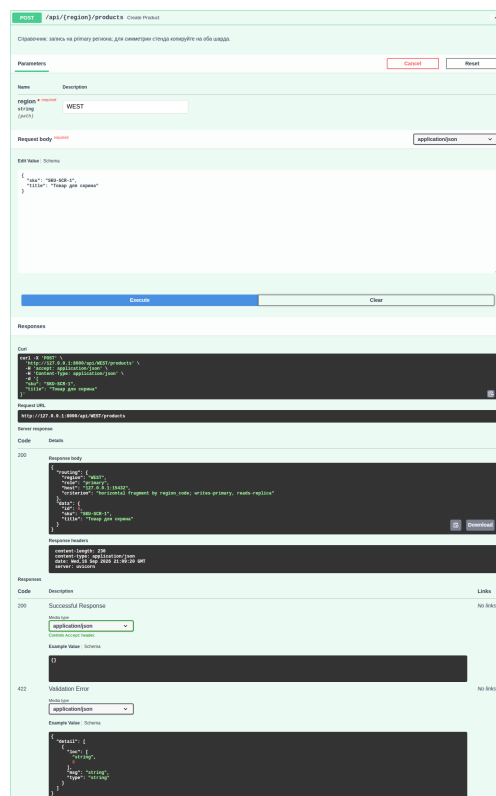


Рисунок 10.4 — 06_swagger_product_post.png

10.5. Создание заявки на отгрузку

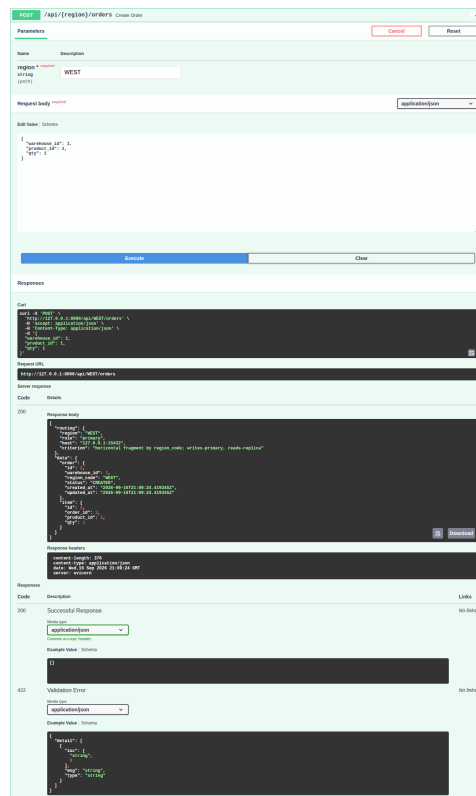


Рисунок 10.5 — 08_swagger_order_post.png

10.6. Цепочка статусов до DELIVERED



Рисунок 10.6 — api_10_status_DELIVERED.png

10.7. Журнал shipment_events

```
GET events — журнал shipment_events

{
  "routing": {
    "region": "WEST",
    "role": "replica",
    "host": "127.0.0.1:15433",
    "criterion": "horizontal fragment by region_code; writes-primary, reads-replica"
  },
  "data": [
    {
      "id": 1,
      "order_id": 1,
      "from_status": null,
      "to_status": "CREATED",
      "created_at": "2026-09-16T21:02:00.482169Z",
      "note": "order created"
    },
    {
      "id": 2,
      "order_id": 1,
      "from_status": "CREATED",
      "to_status": "RESERVED",
      "created_at": "2026-09-16T21:02:00.560929Z",
      "note": "demo RESERVED"
    },
    {
      "id": 3,
      "order_id": 1,
      "from_status": "RESERVED",
      "to_status": "PICKING",
      "created_at": "2026-09-16T21:02:00.595572Z",
      "note": "demo PICKING"
    },
    {
      "id": 4,
      "order_id": 1,
      "from_status": "PICKING",
      "to_status": "SHIPPED",
      "created_at": "2026-09-16T21:02:00.631543Z",
      "note": "demo SHIPPED"
    },
    {
      "id": 5,
      "order_id": 1,
      "from_status": "SHIPPED",
      "to_status": "DELIVERED",
      "created_at": "2026-09-16T21:02:00.670150Z",
      "note": "demo DELIVERED"
    }
  ]
}
```

Рисунок 10.7 — api_11_events.png

10.8. Отказ по критерию контура (чужой регион)

```
Отказ по критерию контура (EAST-WEST)

{
  "detail": "Заявка не найдена на этом фрагменте"
}
```

Рисунок 10.8 — api_12_cross_region.png

10.9. Отказ записи при docker stop west-primary

Отказ записи при docker stop west-primary

```
{
  "detail": {
    "message": "Узел недоступен: region=WEST, role=primary",
    "host": "127.0.0.1:15432",
    "error": "connection failed: connection to server at \"127.0.0.1\", port 15432 failed: Connection refused\n\tIs the server running on that host and accepting TCP/IP connections?",
    "hint": "При отказе primary запись запрещена; попробуйте чтение с replica или восстановите узел (docker start / promote).\"
  }
}
```

Рисунок 10.9 — api_13_write_fail.png

10.10. Чтение с replica при недоступном primary

Чтение с replica при отказе primary

```
{
  "routing": {
    "region": "WEST",
    "role": "replica",
    "host": "127.0.0.1:15433",
    "criterion": "horizontal fragment by region_code; writes=primary, reads=replica"
  },
  "data": [
    {
      "id": 1,
      "warehouse": "WH-WEST-1",
      "sku": "SKU-100",
      "title": "Кабель оптический 1м",
      "qty_available": 100,
      "qty_reserved": 0
    },
    {
      "id": 2,
      "warehouse": "WH-WEST-1",
      "sku": "SKU-200",
      "title": "Блок питания ИБП",
      "qty_available": 50,
      "qty_reserved": 0
    },
    {
      "id": 3,
      "warehouse": "WH-WEST-1",
      "sku": "SKU-300",
      "title": "Коммутатор 24p",
      "qty_available": 20,
      "qty_reserved": 0
    },
    {
      "id": 7,
      "warehouse": "WH-WEST-DEMO",
      "sku": "SKU-DEMO-1",
      "title": "Демо-товар стенда (upd)",
      "qty_available": 22,
      "qty_reserved": 0
    }
  ]
}
```

Рисунок 10.10 — api_13_read_replica.png